

Project Specification Report

Network Simulator

Bhumika Joshi | Neharika Bajaj | Riyanshi | Shristi Muskan

Department of Information Technology
National Institute of Technology, Srinagar

May 24, 2026

1 Introduction

This comprehensive, multi-phase semester project centers on the design, development, and implementation of a comprehensive network simulator. Moving beyond high-level protocol abstractions, the project's objective is to engineer explicit hardware-node abstractions that accurately map granular, cross-layer processes from first principles.

By isolating transmission pathways from data serialization to full process-process communication, this simulator demonstrates the functional bedrock of the OSI model using a deterministic, software-driven architecture rather than relying on pre-existing third-party socket APIs, while maintaining strict procedural decoupling across all protocol layers.

2 Objectives

The primary objective of this project is the construction of a comprehensive, parallel-executing network simulator that authentically emulates functional protocol stack mechanics, rather than presenting layer operations as distinct modular options.

- **Submission 1 (Layers 1 & 2):** Establish core end-device topologies alongside functional hubs, bridges, and switches featuring active MAC address learning and link-layer access control.
- **Submission 2 (Layer 3):** Implement network routing components including classless IPv4 configuration, ARP validation, static route structures, and dynamic RIP table lookups using longest mask matching.

- **Submission 3 (Layers 4 & 7):** Enable process-to-process communication via port allocation, integrate a Go-Back-N sliding window mechanism, and build cooperative FTP and TELNET application layers.

3 Module Description

The architecture is modular, with decoupled files mapping directly to designated networking layers:

- `core.py` & `network_utils.py`: Establishes basic primitives including `IPPacket`, `Frame`, system nodes, addressing structures, and global configuration flags.
- `phy_layer.py`: Manages physical byte-to-bit conversion, formatting of the 8-bit synchronization preamble (10101010), and physical medium bit serialization.
- `datalink.py`: Houses `DataLinkLayer` logic, MAC framing, local unicast/broadcast switching tables, and basic frame-level verification.
- `routing.py` & `network.py`: Controls Layer 3 topology routing, manages local ARP tables, handles cross-subnet gateway checks, and runs the distance-vector routing tables (RIP protocol).
- `transport.py` & `protocols.py`: Implements reliable segment fragmentation and reassembly, assigning abstract port boundaries managed by the Go-Back-N (GBN) state machine.
- `apps.py`: Simulates application environments, containing client/server FTP engines processing filesystem syntax requests (e.g., `GET hello.txt`) and TELNET.

4 Working of the System

Data flows sequentially through a double-traversal pipeline (Down-Stack Encapsulation followed by Up-Stack Decapsulation):

1. **Application Command Initiation:** The client app creates a text command payload and passes it down to the transport layer.
2. **Transport Segment Chunks:** The GBN scheduler fragments payloads into fixed chunk sizes, attaching port targets and sequence metadata.
3. **Network Addressing & Hop Routing:** IP boundaries attach. If the target is localized, it checks ARP cache keys; otherwise, it assigns next-hop gateway targets verified via RIP routing updates.
4. **Framing and Bit Streaming:** Data Link layers add MAC metadata headers. The Physical Layer drops down to raw binary character translation strings prefixed with the synchronization preamble.

5. **Medium Transport and Decoding:** Bits travel through simulated channels to switch/router hubs, where flags like TTL values are decremented, and are subsequently parsed back up through the layers until reaching the receiver.

5 Key Features

- **Physical Bit Encoding:** Converts raw string payloads into structured 8-bit binary ASCII streams for a realistic simulation of wire-level data transfer.
- **Preamble Synchronization:** Prepends a standard 8-bit synchronization pattern (10101010) to every outgoing frame to mark the precise boundary of data transmission.
- **MAC-Based Framing:** Handles encapsulation and decapsulation of higher-layer data with explicit Source and Destination MAC address fields.
- **Unicast MAC Forwarding:** Dynamically builds and updates local switch CAM lookup tables to map MAC addresses to their respective interface ports.
- **Fallback Flooding Logic:** Simulates authentic Layer 2 switch behavior by broadcasting or flooding frames across all active ports when the destination MAC address is unlearned.
- **Stop-and-Wait Flow Control:** Enforces basic link-layer reliability by requiring explicit generation of ACK frames for successfully parsed sequence indices.
- **Decoupled Architecture:** Enforces a clean separation of concerns, ensuring the physical medium remains completely agnostic of the frame data structure it encapsulates.
- **Static and Dynamic Routing Engine:** Supports robust Layer 3 packet forwarding by implementing both static routing configurations and dynamic Distance-Vector routing via the RIP protocol.
- **Longest Mask Matching Optimization:** Simulates realistic IP route lookup choices by evaluating all matching destination prefixes and selecting the tightest network constraint based on subnet mask specificity.
- **Address Resolution Protocol (ARP):** Supports complete broadcast ARP Requests and localized target ARP Replies to cleanly discover and cache neighboring MAC addresses within a local subnet.
- **Isolated Convergence Logging (rip.log):** Isolates dynamic protocol handshakes by trapping runtime Bellman-Ford background modifications and saving clean table updates to an independent database log.
- **End-to-End Simulation Logging (simulation.log):** Intercepts structural stack print calls during live application transfers to write clear, line-by-line encapsulation and decapsulation

steps into a detailed trace file.

6 Assumptions

- Errors are simulated synthetically within the environment.
- No real-time communication or live hardware network card interfaces are used.
- Acknowledgments are assumed reliable and arrive safely without tracking separate ACK losses.

7 Limitations

- No real packet loss simulation or automated timeout recovery states exist.
- The physical layer is simplified to binary character maps without electrical signaling.
- No parallel transmission is supported as functions execute in sequential loops.
- The upward data reception pathway bypasses formal Network Layer parsing.

8 Future Work

- Add a graphical interface to visually show the network topology.
- Implement more protocols like OSPF and Selective Repeat.
- Extend the upward stack pipeline to explicitly process packets at the receiving Network Layer.

9 Conclusions

The discrete network stack simulator successfully models the deterministic execution mechanics governing low-level data transmission within a local broadcast domain.

By implementing explicit abstractions for the Physical and Data Link layers, the simulator visually demonstrates how byte streams undergo binary transformations and preamble attachment before hitting a transmission medium.

Furthermore, the simulation confirms that core properties of Ethernet-based topologies—such as localized MAC address switching tables, dynamic fallback flooding rules, deterministic error boundaries, and reciprocal acknowledgment flows—can be accurately verified using structured software pipelines.

The clear decoupling of the encapsulation states successfully translates textbook theoretical concepts into an observable, scalable debugging architecture.

10 References

1. Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson.
2. Class lecture notes and course materials, Department of Information Technology, National Institute of Technology, Srinagar.
3. *Python Standard Library Documentation — ftplib Module*. Available at: <https://docs.python.org/3/library/ftplib.html>
4. *Python Standard Library Documentation — telnetlib Module*. Available at: <https://docs.python.org/3/library/telnetlib.html>

11 Acknowledgment

The authors acknowledge the use of generative artificial intelligence workflows, specifically Gemini models, during the system optimization stages. These digital assistant architectures were leveraged to structure cross-layer verification tests, clean runtime log presentation syntax, format document schemas, and refine structural project specification reports.